

Lecture 8: YARN (Yet Another Resource Negotiator)

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A

amitkdhar@iitbhlai

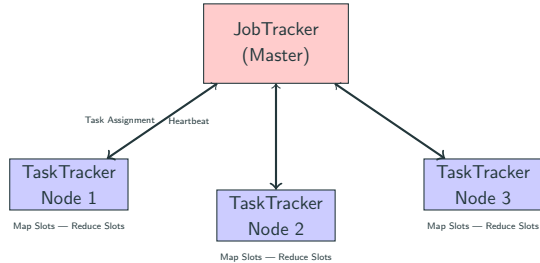
January 22, 2026

Limitations of Hadoop 1.x (MapReduce v1)

Recap: Hadoop 1.x Architecture

- **JobTracker (Master):**
 - Resource Management (tracking nodes, slots).
 - Job Scheduling (assigning tasks).
 - Task Monitoring (re-execution).
- **TaskTracker (Slave):**
 - Execute Map and Reduce tasks.
 - Send heartbeats to JobTracker.

Visualizing Hadoop 1.x Architecture



Limitation 1: The Bottleneck

- **Scalability Issue:**

- The JobTracker does *everything*: resource management AND job lifecycle management.
- Practical limit: 4,000 nodes, 40,000 tasks.

- **Single Point of Failure:**

- If JobTracker fails, all running jobs are lost.
- No high availability in early versions.

Limitation 2: Inefficient Resource Utilization

Static Slot Allocation:

- Resources partitioned into **Map Slots** and **Reduce Slots**.
- Map slots cannot be used for Reduce tasks and vice-versa.

Scenario

If a cluster is running only Map tasks (e.g., ETL), the Reduce slots sit **idle**, wasting cluster capacity.

Limitation 3: Tight Coupling

- Hadoop 1.x was designed specifically for **MapReduce**.
- Other frameworks (Spark, Giraph, MPI) had to:
 - Hack into MapReduce (e.g., Spark on MR).
 - Or maintain a completely separate cluster (operational overhead).

YARN Architecture

Introduction to YARN

- **YARN:** Yet Another Resource Negotiator (Hadoop 2.x+).
- **Core Idea:** Split the two major responsibilities of the JobTracker:
 1. **Resource Management** (Global)
 2. **Job Scheduling/Monitoring** (Per Application)

"YARN is the operating system of Hadoop."

YARN Core Components

1. **ResourceManager (RM):** Global master.
Allocates resources to applications.
2. **NodeManager (NM):** Per-node agent.
Manages containers.
3. **ApplicationMaster (AM):** Per-job master. Negotiates resources from RM, talks to NM to start tasks.
4. **Container:** Abstract unit of resources (CPU + RAM).

Analogy:

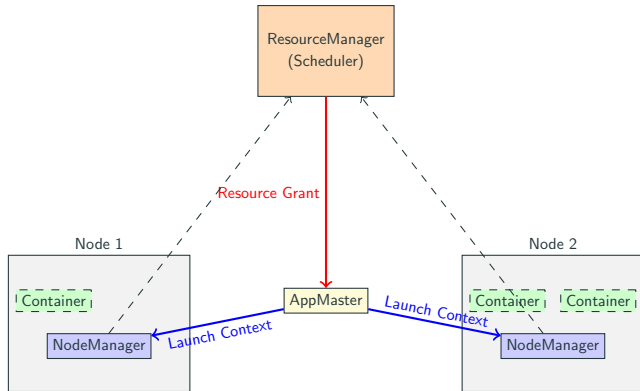
RM = Operating System Kernel

NM = Device Drivers

AM = User Process

Container = Thread/Process Resources

Architecture Diagram



The ResourceManager (RM)

- **Scheduler:**

- Pluggable component (Capacity, Fair).
- Strictly allocates resources.
- Does NOT monitor application status (pure scheduler).

- **ApplicationsManager:**

- Accepts job submissions.
- Negotiates the first container for the ApplicationMaster.
- Restarts the ApplicationMaster on failure.

The NodeManager (NM)

- Runs on every slave node.
- Responsible for:
 - Launching containers.
 - Monitoring resource usage (CPU, Memory) of containers.
 - Reporting node health to ResourceManager.
 - Killing containers if instructed by RM or AM.

The ApplicationMaster (AM)

- Framework-specific library (MapReduce AM, Spark AM).
- Runs inside a container.
- **Responsibilities:**
 1. Ask RM for resource containers via heartbeat.
 2. Instruct NMs to start tasks in acquired containers.
 3. Monitor task progress and handle task failures.

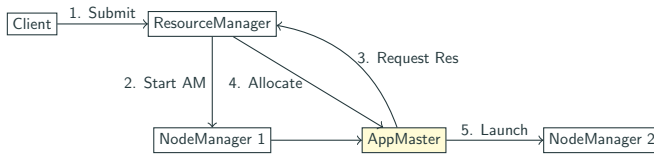
- Represents a grant of allocated resources (e.g., 2GB RAM, 1 vCore).
- **Dynamic:** Unlike MRv1 slots, containers can be of any size (within limits).
- Tied to a specific node.
- Used to run:
 - The ApplicationMaster itself.
 - MapReduce Tasks.
 - Spark Executors.

YARN Application Workflow

Step-by-Step Execution Flow

1. Client submits application to RM.
2. RM starts the ApplicationMaster (AM) on a NodeManager.
3. AM registers with RM.
4. AM negotiates resource containers from RM.
5. AM talks to NMs to launch containers.
6. Containers execute the user code (Map/Reduce/Spark).
7. AM monitors progress.
8. On completion, AM unregisters and exits.

Visualizing the Workflow



Skeletal Code: Application Submission

```
1  // 1. Create YARN Client
2  YarnClient yarnClient = YarnClient.createYarnClient();
3  yarnClient.init(conf);
4  yarnClient.start();
5
6  // 2. Create Application
7  YarnClientApplication app = yarnClient.createApplication();
8  GetNewApplicationResponse appResponse = app.getNewApplicationResponse();
9
10 // 3. Define AM Launch Context
11 ContainerLaunchContext amContainer = Records.newRecord(ContainerLaunchContext.class);
12 amContainer.setCommands(Collections.singletonList(
13     "$JAVA_HOME/bin/java com.example.MyApplicationMaster"
14 ));
15
16 // 4. Submit
17 ApplicationSubmissionContext appContext = app.getApplicationSubmissionContext();
18 appContext.setAMContainerSpec(amContainer);
19 yarnClient.submitApplication(appContext);
```

Client Submission Logic

Skeletal Code: ApplicationMaster Logic

```
1  // AMRMClientAsync handles communication with RM
2  AMRMClientAsync<AMRMClient.ContainerRequest> rmClient =
3      AMRMClientAsync.createAMRMClientAsync(1000, new AMRMClientAsync.CallbackHandler() {
4          @Override
5          public void onContainersAllocated(List<Container> containers) {
6              for (Container c : containers) {
7                  // Launch task on allocated container
8                  nmClient.startContainerAsync(c, ctx);
9              }
10         }
11     });
12
13 rmClient.init(conf);
14 rmClient.start();
15
16 // Request 2 containers with 1GB memory, 1 vCore
17 ContainerRequest containerAsk = new ContainerRequest(
18     Resource.newInstance(1024, 1), null, null, Priority.newInstance(0));
19 rmClient.addContainerRequest(containerAsk);
```

AM Resource Request

Scheduling Algorithms

The Scheduling Problem

Context:

- Multi-tenant cluster (Marketing, Engineering, Data Science).
- Finite resources (RAM, CPU).
- How do we decide which application gets resources *right now*?

Three Main Schedulers:

1. FIFO Scheduler.
2. Capacity Scheduler (Default in Cloudera/Hadoop 2).
3. Fair Scheduler (Default in CDH/Spark usually).

1. FIFO Scheduler

- **Logic:** First In, First Out.
- Jobs are placed in a single queue.
- **Pros:** Simple, efficient for single-user.
- **Cons:** Large jobs block small jobs ("Head of line blocking").

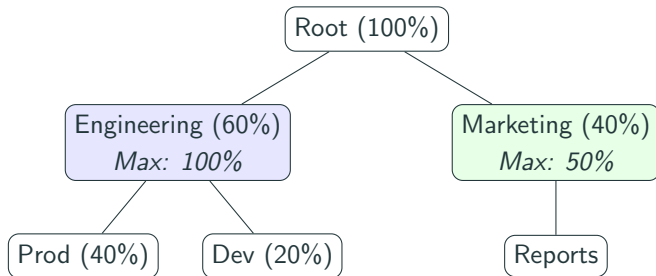


2. Capacity Scheduler

Philosophy: Dedicated "lanes" for different teams to ensure guarantees.

- Cluster resources are divided into **Queues**.
- Each queue has a specific **capacity** (e.g., Eng: 60%, Mkt: 40%).
- **Elasticity:** Queues can use idle resources from other queues (up to a max cap).

Capacity Scheduler Visualization



Configuring Capacity Scheduler

capacity-scheduler.xml

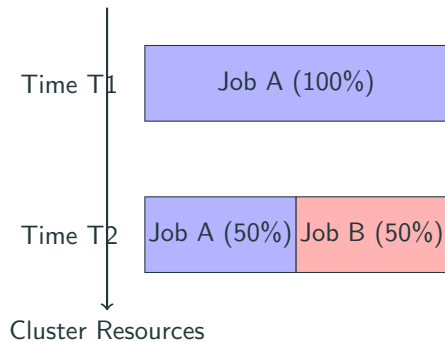
```
1 <property>
2   <name>yarn.scheduler.capacity.root.queues</name>
3   <value>engineering,marketing</value>
4 </property>
5
6 <property>
7   <name>yarn.scheduler.capacity.root.engineering.capacity</name>
8   <value>60</value>
9 </property>
10
11 <property>
12   <name>yarn.scheduler.capacity.root.marketing.capacity</name>
13   <value>40</value>
14 </property>
```

3. Fair Scheduler

Philosophy: All running jobs get an roughly equal share of resources over time.

- No rigid capacity reservations needed (though possible).
- **Dynamic:**
 - 1 Job running → It gets 100% resources.
 - 2nd Job starts → Resources split 50% / 50%.
- Uses "Pools" instead of "Queues".

Fair Scheduler Visualization



Resources are preempted or released to balance the share dynamically.

Configuring Fair Scheduler

fair-scheduler.xml

```
1 <allocations>
2   <pool name="production">
3     <minResources>1024 mb, 1 vcores</minResources>
4     <weight>2.0</weight>
5   </pool>
6
7   <pool name="test">
8     <weight>1.0</weight>
9   </pool>
10
11   <!-- Default policy -->
12   <defaultQueueSchedulingPolicy>fair</defaultQueueSchedulingPolicy>
13 </allocations>
```

YARN in the Ecosystem

YARN enables **Data Operating System** capabilities.

- **MapReduce:** Batch processing.
- **Spark:** In-memory data processing.
- **Tez:** DAG execution engine (powers Hive).
- **Storm/Flink:** Real-time streaming.
- **HBase:** NoSQL (uses YARN for some operational tasks, though mostly independent).

All run simultaneously on the SAME cluster hardware.

YARN High Availability (HA)

- **ResourceManager HA:**

- Active/Standby architecture (using ZooKeeper).
- If Active RM fails, Standby takes over.
- State saved in "State Store" (FileSystem or ZooKeeper).

- **Work Preservation:**

- Applications continue running during RM restart/failover.
- NMs simply report to the new Active RM.

How do users request resources?

- **Memory (RAM):**

- 'yarn.nodemanager.resource.memory-mb' (Total NM RAM).
- 'yarn.scheduler.minimum-allocation-mb' (Min container size).

- **CPU (vCores):**

- Virtual Cores, not physical cores.
- 'yarn.nodemanager.resource.cpu-vcores'.

Questions?